

Performance Tuning Checklist

0	Database Partitioning - Ensures database performance remains constant over time - CM tables holding time-series data are be partitioned by time-range Docs: Database Partitioning	- Increase process intake throughput - Reduce page loading latency	Database setup
1	Plan for removing/detaching old partitions - In PostgreSQL, too many partitions may cause severe database performance degradation. - Around 100-300 partitions across the whole database seems to work. More than that may result in issues. At 100 partitions for a particular table, performance may start to degrade. - Read Performance issues with PostgreSQL partitioning - Deleting or detaching old partitions that fall out of the data retention policy helps keep the overall number low.	- Keeps database healthy - Increases overall CM performance	Database setup
2	"Is event update" time window - Limit how far in the past CM looks for the original event for an event to be considered update - This will bring performance improvements for long running projects if the project has event updates and is configured according to the above partitioning recommendations - If your project doesn't have updates at all, ignore them by setting ignore_event_updates to true, as specified here. You will benefit from an enormous performance boost. - Configuration parameter (in seconds): <code>update_time_frame_secs</code> , that is: config.json Expand source <pre>{ ... "event": { ... "update_time_frame_secs":, ... } }</pre>	Increase process intake throughput	Core config file: <code>config.json</code>

	<pre> }, } }</pre>		
	Docs: Core configuration file		
3	Alerts Page's time window • Limit how far in the past CM will fetch alerts to populate the Alerts (landing) page. • This will bring performance improvements for long running projects if the project is configured according to the above partitioning recommendations • This is configured as a Alerts page's quick filter (at <code>gui-config.json</code> file). That is: gui-config.json Expand source <pre>{ ... "configurationInstance": { "pages": { "alerts": { "type": "listing", "title": "alerts.title", "config": { ... "table": { ... "quickFilters": [... { "key": "timestamp", "i18n": true, "type": "select", "multi": false, "filter": "gteq", "options": [{ "label": "placeholder.all.time" }, { "label": "placeholder.last.day", "value": { "type": "subtractTime", "unit": "day", "value": 1</pre>	• Reduce page load latency	UI config file: • <code>gui-config.json</code>

```

    }, {
      "label":
"placeholder.last.week",
      "value":
{
  "type": "subtractTime",
  "unit": "week",
  "value": 1
}
    }, {
      "label":
"placeholder.last.month",
      "value":
{
  "type": "subtractTime",
  "unit": "month",
  "value": 1
}
    }, {
      "label":
"placeholder.last.year",
      "value":
{
  "type": "subtractTime",
  "unit": "year",
  "value": 1
}
    }, {
      "label":
"placeholder.last.five.years",
      "value":
{
  "type": "subtractTime",
  "unit": "year",
  "value": 5
}
  }],
  "cssClass":
"fdz-js-alerts-table-date-filter",
  "renderType": "label",
  "placeholder": "placeholder.timestamp",
  "defaultValue": {
    "label":
"placeholder.last.five.years"
  }
},
...

```

	} Docs: Fields		
4	Search Page's time window • Limit how far in the past CM will fetch events to populate the Search page. • This will bring performance improvements for long running projects if the project is configured according to the above partitioning recommendations • This is configured as the Search page's default time range for date picker (at <code>gui-config.json</code> file). That is: gui-config.json Expand source { ... "configurationInstance": { ... "event-search": { "type": "search", "title": "event-search.title", "config": { "form": { "title": "search.button", "fields": [{" "type": "range", "title": "event.timestamp", "fields": [{ "key": "schema.fields.timestamp", "i18n": true, "type": "datepicker", "width": "50%", "filter": "gteq", "defaultVal": { "type": "subtractTime", "unit": "year", }	• Reduce page load latency	UI config file: • <code>gui-config.json</code>

• Reduce page load latency

	<pre>"value": 5 }, "placeholder": "form.placeholder.from" }, { "key": "schema.fields.timestamp", "i18n": true, "type": "datepicker", "width": "50%", "filter": "\lteq", "placeholder": "form.placeholder.to" } }</pre>		
	Docs: Configuring the default value of a datepicker		
5	Transaction History's time windows (Details Page) • Limit how far in the past CM will fetch events to populate the Details page's Transaction History. • This will bring performance improvements for long running projects if the project is configured according to the above partitioning recommendations • This is configured as a Alerts page's quick filter (at <code>gui-config.json</code> file). That is: gui-config.json Expand source <pre>{ ... "configurationInstance": { ... "pages": { "event-details": { "type": "details", "title": "event- details.title", "config": { "widgets": [{ "key": "transaction-history", "name": "history", "type": "entity- history",</pre>	• Reduce page load latency	UI config file: • <code>gui-config.json</code>

<pre>"title": "widget.linked_events.title", "width": "100%", "cssClass": "fdz-css-entity-history-widget", "resource": "event", "iconCssClass": "icon-transaction", "checkboxLabel": "placeholder.match-all-fields", "timestampSelectConfiguration": { "key": "timestamp", "i18n": true, "type": "select", "multi": false, "filter": "gteq", "options": [{ "label": "placeholder.all.time" }, { "label": "placeholder.last.day", "value": { "type": "subtractTime", "unit": "day", "value": 1 } }, { "label": "placeholder.last.week", "value": { "type": "subtractTime", "unit": "week", "value": 1 } }, { "label": "placeholder.last.month", "value": { "type": "subtractTime", "unit": "month", "value": 1 } }] } }</pre>		
---	--	--

	<pre> }, { "label": "placeholder.last.year", "value": { "type": "subtractTime", "unit": "year", "value": 1 } }, { "label": "placeholder.last.five.years", "value": { "type": "subtractTime", "unit": "year", "value": 5 } }], "cssClass": "fdz-js-transaction-history-date-filter", "renderType": "label", "placeholder": "placeholder.timestamp", "defaultValue": { "label": "placeholder.last.five.years" } }, } </pre> Docs: Entity History		
6	Event Review Next's time window • Limit how far in the past CM will look for the next event to be reviewed • This will bring performance improvements for long running projects if the project is configured according to the above partitioning recommendations Docs: Review Next	• Reduce page load latency	ARNO config file: • review-next-order-config.json
7	Case Review Next's time window • Limit how far in the past CM will look for the next case to be reviewed	• Reduce page load latency	ARNO config file: • review-next-order-config.json

	• This will bring performance improvements for long running projects if the project is configured according to the above partitioning recommendations Docs: Review Next		
8	Upper-bound the COUNT of event listing pages • Limit the count of results retrieved by an event pagination request • This avoids a full table scan to fetch the count, which could be prohibitively expensive and massively slow down event listings/searches Docs: Events Count Overhead	• Reduce page load latency	UI config file: • <code>gui-config.json</code>
9	Disable enrichers that are not required on alerts/search page • Enriching can be expensive when applied to a list of events, be sure they are being used if and only if you need the enriching data to be displayed in the listing page. Otherwise, enrichments should be disabled Docs: Core (config.json) (Check <code>ignore_on_list</code> parameter)	• Page load latency	Core config file: • <code>config.json</code>
10	Alerts/Search page configuration must refer the correct timestamp field • Event listing pages that refer to event's timestamp, either in page layout (columns) or page queries (filter), must be configured to refer the event's core <code>timestamp</code> field, and not to any other fields that may also holding the events timestamp, such as <code>schema.fields.timestamp</code>. • Miss-configuring this will prevent CM from be able to take part of database partitioning, as <code>timestamp</code> is the schema partitioning field. Check example below: gui-config.json Expand source <pre> { ... "configurationInstance": { "pages": { "alerts": { "type": "listing", "title": </pre>	• Reduce page load latency	UI config file: • <code>gui-config.json</code>

	<pre>"alerts.title", "config": { "table": { "filter": "'alert' eqb 'true' AND 'statusId' eq 'unknown'", "linkTo": "event", "columns": [{ "key": "timestamp", // this should be "timestamp" and not "schema.fields.timestamp" "type": "timestamp", "title": "event.timestamp", "cssClass": "fdz-css-align-left fdz- css-text--xsmall fdz-css-lh--large", "sortable": true }], ... }</pre>		
11	Fine-tuning parameters in config.json • <code>config.json</code> allows to tweak some parameters regarding CM internals, such as: ◦ The number of worker threads to process the events arriving from Pulse. ◦ The size of CM's internal buffer to hold the events (pre)fetched from EVENT queue. ◦ The threshold of events to be stored in batch. ◦ The timeout to trigger the storage of events in batch. ◦ The limit threshold of retrieved events from listing queries. • These parameters can be changed via <code>config.json</code> file (CM restart needed): config.json Expand source <pre>{ ... "performance": { "event_worker_threads": 8, "queue_prefetch_size": 200, "batch_storage_size": 300, "batch_storage_timeout": 1000</pre>	• Increase process intake throughput • Reduce page loading latency	Core config file: • <code>config.json</code>

	<pre> } } </pre> Docs: Core (config.json)		
12	Postgres specific tuning CM works best with the following PostgreSQL parametrization.	- Increase process intake throughput - Reduce page loading latency	Database setup
13	Dedicated database connection pool for reports Allows to have a dedicated database connection pool to generate reports, so we can segregate the reports' queries from all other CM queries. Making it possible to have a different timeout policy for those queries.	Dedicated resources to generate reports (heavy task)	Core config file: <code>config.json</code>
14	Limit the amount of database queries when listing events through Alerts/Search pages - If you don't require events to be enriched with data from AM_EVENT_STORAGE, disable fetch_with_event_storage_enricher. This will speed up the fetching of events. - Docs: Core (config.json) Expand source <pre> { "event": { "listing": { "fetch_with_event_storage_enricher": false } } } </pre>	Reduce page loading latency by reducing the number of database queries required to serve the page	Core config file: <code>config.json</code>
15	Read-Only Replica Database - Allows to redirect CM's UI event-based requests, such as Alerts, Events, and the Details page's Transaction History to a read-only database (ie. a read-only replica of CM's main database). - Rational is to prevent CM's (main) database from being overwhelmed with user activity requests while it is already processing events arriving from Pulse. - Docs: Read-Only Replica Database	Reduce contention in database by scaling it with a read-only replica	Core config file: <code>config.json</code> <code>gui-config.json</code>
16	Avoid large table scans on the Case Linked Events widget (Postgres specific) This widget is typically configured to display the Case Linked Events sorted by event date; however, on large volume deployments, configuring the widget to use only this field may lead to poor query performance. In	Linked events retrieval reduced from 90 seconds to 54 ms (on a particular deployment).	UI config file: <code>gui-config.json</code>

	order to prevent this, please add the eventId field to the sorted fields in gui-config.json, i.e.: Expand source <pre>... "orderBy": "timestamp desc, id asc", "orderByFallback": ["timestamp desc, id asc"], ...</pre>		
17	Configure the Notifiers feature for Insights If you use Insights, then data should be sent to it via a message broker, using Case Manager's external notifications feature. Then, you can use Insights' CM Logstash pipelines to ingest the data from the message broker. This increases CM's overall throughput significantly. This can be configured in config.json, by adding the following: Expand source <pre>"notifications": { "enabled": true, "queue_size": 100000, "pool_size": 1, "cool_down_period_millis": 1000 }</pre> Read more about this configuration here.	Increase in event throughput	Configuration: config.json
18	Configure the batches - Case Manager offers a series of batches used to perform a multitude of tasks, such as: - Persisting events to AM_EVENT_STORAGE (or all event tables when ignore_event_updates is true - see performance configuration): event_storage_batch - Persisting automation rule actions: automation_actions_batches - Persisting event explanations: batch_storage_size and batch_storage_timeout - Persisting entries to AM_AUDIT: audit_batch - Executing background healthcheck tasks: healthcheck_worker_threads and max_queued_healthchecks All of these properties can be configured in the performance property of the config.json file. For a	- Decrease in database load - Increase in event throughput	Configuration: config.json

	better performance, ensure you configure these properties according to your requirements. Expand source <pre>"performance": { "batch_storage_size": 300, "batch_storage_timeout": 1000, "event_storage_batch": { "queue_size": 500, "batch_size": 500, "batch_timeout": 1000 }, "automation_actions_batches": { "queue_size": 500, "batch_size": 500, "batch_timeout": 1000 }, "audit_batch": { "queue_size": 250, "batch_size": 150, "batch_timeout": 1000 }, "healthcheck_worker_threads": 6 "max_queued_healthchecks": 12 }</pre> Read more about this configuration here.		
19	Configure CM to ignore event updates, if possible. • If your project does not require Case Manager to process event updates, ensure that ignore_event_updates is set to true • This allows CM to make a few, very valuable optimizations: ◦ Disable the query made, for each event that arrives, that checks whether that event is an update. ◦ Persist events using batches, thus increasing throughput. Note that the configuration for this batch is defined by the event_storage_batch property. This property can be configured in the performance property of the config.json file: Expand source <pre>"performance": { ... "ignore_event_updates": true, ... },</pre>	• Increase in event throughput	Configuration: • <code>config.json</code>
20			

	Consider using the state machine's <code>initial_state</code> property to set the initial state of events, instead of relying on automation rules. The <code>initial_state</code> property of the configuration of state machines allows defining a state that will always be set for new events. The way this is done is heavily optimized and, as such, is significantly cheaper than doing it via automation rules. This property can be configured in the <code>state_machines</code> property of the <code>config.json</code> file: Expand source <pre>"state_machines": [{ ... "key": "status", "initial_state": "PULSE", ... }],</pre>	- Increase in event throughput - Reduction of database load	Configuration: <code>config.json</code>
21	Set timestamp field as mandatory in the search page. Having the timestamp as a required field in the search page prevents users from performing searches over the entire event storage, which can be very expensive and detrimental to performance. Docs: Search/Listing Page Form Schema Fields	- Faster searches - Reduction of database load	UI config file: <code>gui-config.json</code>
22	Set maximum number of allowed filters in the search page. Having the <code>maxFilters</code> property set in the search configuration prevents users from performing searches with a significant amount of fields, which can be very expensive and detrimental to performance. Docs: Search/Listing Page Form Schema Fields	- Faster searches - Reduction of database load	UI config file: <code>gui-config.json</code>
23	Disable automatic search on load. - Disabling the automatic search that is performed when the search page is opened by a user can be achieved via the <code>searchOnLoad</code> property in the search configuration. - This reduces a lot of load by reducing the number of searches made. Docs: Search/Listing Page Form Schema Fields	Reduction of database load	UI config file: <code>gui-config.json</code>
24	Delay event reads Ensure that, if the event storage listing is active, the event retrieval should not include the immediately stored events (in <code>AM_EVENT</code>) but should be delayed	Event display correctness	Configuration: <code>config.json</code>

	as the AM_EVENT_STORAGE may have not been written. This property can be configured in the listing property under event_properties of the config.json file: <pre> { "event_properties": { "listing": { "delay_event_reads": true } } } </pre>		
25	Limit the amount of database queries when exporting events - If you don't require events to be enriched with data from AM_EVENT_STORAGE, disable fetch_with_event_storage_enricher. This will speed up the event export generation. - Docs: Export Resources (Check <code>fetchWithEventStorageEnricher</code> parameter) gui-config.json Expand source <pre> { "exportButton": { "resource": "eventReport", "config": { "fetchWithEventStorageEnricher": false } } } </pre>	Improve event export functionality performance by reducing the number of database queries required to generate the export	UI config file: <code>gui-config.json</code>
26	Compress event storage data before insertion in the database - If disk usage is an issue, enabling compress_event_data can help. This makes event storage data be compressed before insertion in AM_EVENT_STORAGE. - Docs: Event Storage Configuration Reference (Check <code>compress_event_data</code> parameter)	Less disk usage for events that are compressed before ingestion	Configuration: <code>config.json</code>